

# Parallel Implementation of Image Recreation and Rendering with a Genetic Algorithm

Mohammed Elkoumy  
elkoum@rpi.edu  
Rensselaer Polytechnic Institute  
USA

Mahir Mobarrat  
mobarm@rpi.edu  
Rensselaer Polytechnic Institute  
USA

Quinten Pienaar  
pienaq@rpi.edu  
Rensselaer Polytechnic Institute  
USA

Ganwar Harsh  
gangwh@rpi.edu  
Rensselaer Polytechnic Institute  
USA



Figure 1: Mona Lisa reconstruction results using triangle-based genetic optimization.

## Abstract

Image vectorization transforms a raster image into a vector representation composed of geometric primitives such as triangles or polygons. Classical approaches such as Potrace [9] fit Bézier paths to segmented regions, while differentiable rendering methods such as LIVE [6] and SuperSVG [3] frame vectorization as a layered optimization problem. Both families impose structural assumptions

and require differentiable pipelines, limiting applicability when the objective is discontinuous or primitive topology is unconstrained.

Genetic algorithms (GAs) sidestep these limitations by treating vectorization as a black-box optimization problem [1, 2]. Their population-based structure exposes natural data-level parallelism—each chromosome can be evaluated independently.

We present a fully parallel, multi-level GA system for triangle-based image vectorization in which each chromosome encodes vertex coordinates and RGBA channels for a configurable number of semi-transparent triangles. Fitness is evaluated under five loss functions targeting distinct perceptual failure modes, accelerated by batched CUDA kernels that rasterize and score the entire population in a single multi-GPU launch. An MPI island model distributes independent populations across cluster nodes with periodic ring migration to sustain diversity. Experiments on both commodity workstations and IBM POWER9 HPC nodes demonstrate substantial

**Unpublished working draft. Not for distribution.**

Permission to make digital or hard copies of all or part of this work for personal or internal use, or the internal or personal use of specific clients, is granted by ACM for users registered with ACM, provided that the fee of \$12.00 is paid directly to ACM. This permission is granted on the condition that the copier pay the stated fee per copy to the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RPI '26, Troy, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

throughput gains over serial baselines and reconstruction quality unattainable under serial resource constraints. The implementation is available at: <https://github.com/mohammed-elkomy/Proj-4320>

## Keywords

Genetic Algorithms, Parallel Computing, Image Vectorization, CUDA, MPI, Evolutionary Optimization

## ACM Reference Format:

Mohammed Elkoumy, Quinten Pienaar, Mahir Mobarrat, and Ganwar Harsh. 2026. Parallel Implementation of Image Recreation and Rendering with a Genetic Algorithm. In *Proceedings of Rensselaer Polytechnic Institute (RPI '26)*. ACM, New York, NY, USA, 10 pages.

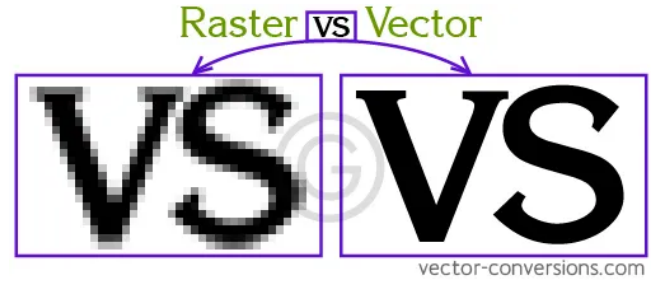
## 1 Introduction

Many real-world problems require recovering, compressing, or representing visual data in a usable form. Image reconstruction can be performed either in raster (pixel-based) or vector (geometry-based) form. Raster-based reconstruction provides high pixel-level fidelity but produces fixed-resolution outputs where scaling introduces blur or aliasing, and storing large collections of reconstructions incurs a substantial memory footprint. Vector-based reconstruction offers resolution-independent representations, compact encodings, and integrates naturally with geometry-based graphics pipelines, enabling efficient rendering and manipulation [9].

Vector-based reconstruction requires solving a challenging optimization problem: approximate a target image using a set of parameterized geometric primitives, each with jointly optimized geometry, color, and opacity. The rendering pipeline introduces discontinuities at primitive boundaries (triangle edges, polygon contours) that render the objective non-differentiable almost everywhere, defeating classical gradient-based solvers unless the pipeline is explicitly reformulated [5]. Classical approaches such as Potrace [9], trace contours and fit Bézier paths to segmented regions. More recent methods such as LIVE [6] and SuperSVG [3] frame vectorization as a layered differentiable rendering problem. Both families, however, impose strong structural assumptions such as contour topology or layer ordering that limit generalization to arbitrary target images.

Evolutionary algorithms offer a complementary approach that makes no assumptions about differentiability or problem structure. By maintaining a population of candidate solutions and iteratively applying selection, crossover, and mutation, Genetic Algorithms (GAs) can optimize directly over discontinuous rendering objectives [1, 2]. Early demonstrations of evolutionary image synthesis date to Sims [10], who evolved images through mathematical expression trees, and to Johansson [4], who approximated paintings using small sets of semi-transparent polygons. These works established that visually faithful reconstructions are attainable with a limited number of geometric primitives, provided the optimizer is given sufficient generations to explore the search space.

The population-level structure of GAs naturally exposes data-level parallelism: every chromosome's fitness is independent of all others within a generation, so the entire population can in principle be evaluated simultaneously on massively parallel hardware. Prior work has explored GPU-accelerated GAs in combinatorial optimization [11] and neural architecture search [8], yet the specific problem of GPU-parallel triangle-based image vectorization at cluster scale



**Figure 2: Raster vs. vector representation. The raster image (left) appears pixelated, while the vector representation (right) remains sharp due to its resolution independent geometric description.**

remains underexplored. Existing implementations either execute serially [4], offload only partial computation to the GPU, or do not report scaling behavior beyond a single node.

In this work, we implement and analyze a fully parallel GA system for triangle-based image vectorization spanning thread-level CUDA parallelism, multi-GPU distribution, and multi-node MPI execution under an island model. We evaluate reconstruction quality under five loss functions designed to capture distinct perceptual failure modes, and we profile the system on both commodity workstations and AIMOS IBM POWER9 HPC nodes (V100 GPUS) to characterize the scaling behavior of evolutionary image reconstruction at cluster scale.

## 2 Problem Statement

### 2.1 Image Approximation as Optimization

Let  $I_{\text{ref}} \in [0, 1]^{H \times W \times 3}$  be a target RGB image of height  $H$  and width  $W$ . The goal is to find a configuration of  $N$  semi-transparent triangles whose rendered composite  $I_{\text{render}}$  minimizes a perceptual dissimilarity measure with respect to  $I_{\text{ref}}$ .

Each triangle  $t \in \{1, \dots, N\}$  is parameterized by a tuple comprising three vertex coordinates in normalized image space and an RGBA color vector5:

$$\theta_t = (x_0^t, y_0^t, x_1^t, y_1^t, x_2^t, y_2^t, r^t, g^t, b^t, \alpha^t) \in [0, 1]^{10}.$$

The full state vector  $\mathbf{x} = (\theta_1, \dots, \theta_N) \in [0, 1]^{10N}$  is treated as a single real-valued chromosome of length  $10N$  by the optimizer3.

### 2.2 Rendering Model

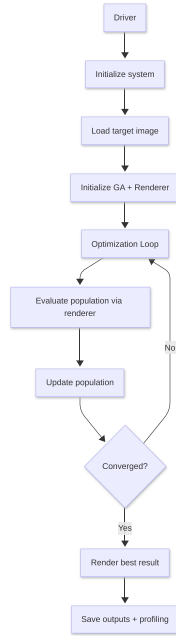
Triangles are composited in layer order using the Porter–Duff *over* operator [7]. Starting from a white background canvas, each triangle  $t$  updates every pixel  $(i, j)$  it covers4:

$$C^{(t)}(i, j) = C_{\text{src}}^{(t)} \alpha^t + C^{(t-1)}(i, j) (1 - \alpha^t),$$

for pixels within triangle  $t$ , and  $C^{(t)}(i, j) = C^{(t-1)}(i, j)$  otherwise. The final rendered image is  $I_{\text{render}} = C^{(N)}$ .

### 2.3 Fitness Functions

Because the rendered objective is non-differentiable at triangle boundaries, fitness is measured through one of three loss functions, each targeting a distinct perceptual failure mode.



**Figure 3: High-level workflow of the system.** The driver initializes the configuration and target image, and iteratively evolves the population using the Genetic Algorithm (GA). At each iteration, candidate solutions are evaluated through the rendering pipeline until convergence, after which the best result is rendered and saved.

**Mean Squared Error (MSE).** The standard per-channel squared residual:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{HW} \sum_{i,j} \|I_{\text{ref}}(i,j) - I_{\text{render}}(i,j)\|_2^2.$$

MSE treats all pixels uniformly and provides a balanced baseline signal.

**Binary Cross-Entropy (BCE).** Treating each rendered pixel channel as a probability in  $[0, 1]$ :

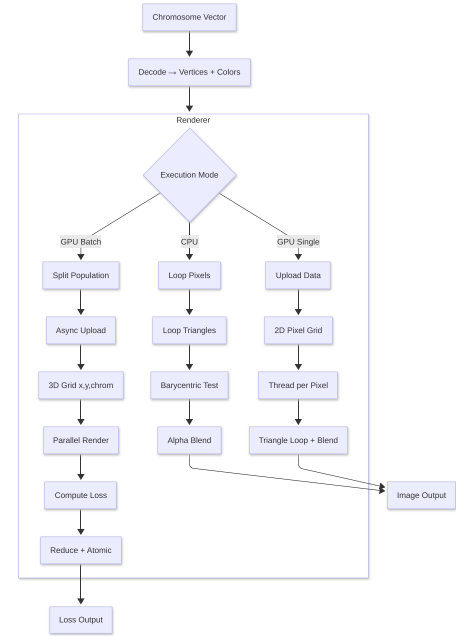
$$\mathcal{L}_{\text{BCE}} = -\frac{1}{HW} \sum_{i,j,c} \left[ I_{\text{ref},c} \log(I_{\text{render},c} + \varepsilon) + (1 - I_{\text{ref},c}) \log(1 - I_{\text{render},c} + \varepsilon) \right],$$

where  $\varepsilon$  is a small constant for numerical stability. BCE penalizes confident wrong predictions more aggressively than MSE, particularly near channel extremes.

**Area-Weighted MSE (WMSE).** MSE scaled by the total normalized triangle area:

$$\mathcal{L}_{\text{WMSE}} = \mathcal{L}_{\text{MSE}} \cdot \left( \sum_{t=1}^N A_t + \varepsilon \right)^\gamma,$$

where  $A_t$  is the area of triangle  $t$  in normalized  $[0, 1]^2$  space and  $\gamma > 0$  is a tunable exponent. This penalty discourages configurations that rely on a few large triangles, encouraging finer-grained coverage.



**Figure 4: Rendering and evaluation pipeline.** A chromosome is decoded into geometric primitives (vertices and colors), which are processed through different execution modes: CPU rendering, single-GPU rendering, and batched multi-GPU evaluation. The CPU and single-GPU paths produce rendered images, while the batched GPU path computes fitness values directly using parallel rendering and reduction operations.

## 2.4 Optimization Problem

The vectorization task is a continuous black-box minimization:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in [0,1]^{10N}} \mathcal{L}(I_{\text{ref}}, \mathcal{R}(\mathbf{x})),$$

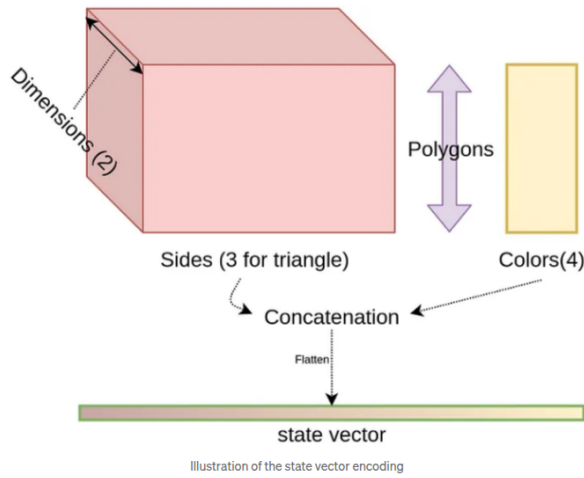
where  $\mathcal{R}(\mathbf{x})$  denotes the rendering operator and  $\mathcal{L}$  is any of the three loss functions above. No gradient of  $\mathcal{L}$  with respect to  $\mathbf{x}$  is assumed or computed; the optimizer interacts with the objective solely through function evaluations.

## 3 Methodology

### 3.1 Genetic Algorithm

We solve the black-box minimization of Section 2 with a steady-state Genetic Algorithm operating on a population of  $P$  real-valued chromosomes, each of length  $10N$ . The algorithm proceeds in discrete generations; each generation applies selection, reproduction, and replacement to drive the population toward lower-loss configurations.

**Initialization.** Each chromosome is initialized by sampling all  $10N$  parameters independently from  $\mathcal{U}(0, 1)$ , except the alpha channel of every triangle, which is fixed to a small constant  $\alpha_{\text{init}}$  to prevent early generations from being dominated by opaque, randomly colored triangles. The fitness of every chromosome is evaluated before the first generation begins.



**Figure 5: Encoding of the state vector used by the Genetic Algorithm (GA).** Each chromosome is represented as a flattened vector of parameters encoding triangle geometry and appearance. The vector is partitioned into vertex coordinates and color channels, which are decoded into geometric primitives (triangles) before rendering.

**Selection.** At the start of each generation the population is sorted in ascending order of fitness loss. The top  $P - D$  chromosomes are retained as elites, where  $D$  is a configurable discard count (default  $D = \lceil 0.75P \rceil$ ). The bottom  $D$  slots are replaced by offspring produced through crossover or mutation.

**Crossover.** With probability  $p_c$  a discarded slot is filled by crossing two parents drawn uniformly at random from the full population. Two crossover strategies are employed:

- *Single-point crossover:* a random triangle index  $k$  is chosen; the child inherits triangles  $1, \dots, k$  from parent 1 and triangles  $k+1, \dots, N$  from parent 2.
- *Uniform crossover:* each vertex independently inherits its coordinates from one of the two parents with equal probability.

The two strategies are selected with equal probability when crossover is applied.

**Mutation.** With probability  $1 - p_c$  a discarded slot is filled by mutating a single chromosome drawn from the population. Two mutation strategies are used:

- *Perturbation mutation:* each coordinate is displaced by a random perturbation drawn from  $\mathcal{U}(-\delta, \delta)$  with probability 0.25 per gene, where  $\delta$  is a generation-adaptive scale; out-of-bounds values are reflected back into  $[0, 1]$ .
- *Resampling mutation:* selected genes are replaced by a fresh sample from  $\mathcal{U}(0, 1)$ , providing large jumps to escape local minima.

Perturbation mutation is applied with probability 0.95; resampling is applied otherwise.

**Termination.** The GA halts when either a maximum generation budget  $G_{\max}$  is reached or a stagnation criterion fires: if the best loss does not improve by more than a threshold  $\tau$  (absolute or relative)

for  $S$  consecutive generations, the run is considered converged and terminates early.

### 3.2 GPU-Accelerated Fitness Evaluation

The dominant cost per generation is fitness evaluation: every chromosome must be rasterized against the target image and its loss computed. In a serial implementation this requires  $P$  sequential render passes, each touching  $H \times W$  pixels and  $N$  triangles. We accelerate this step entirely on the GPU using CUDA, eliminating the serial bottleneck.

**Batched kernel design.** All  $P$  chromosomes are evaluated in a single batched CUDA kernel launch. The three-dimensional grid is organized as

$$\text{grid} = (\lceil W/B \rceil, \lceil H/B \rceil, P),$$

where  $B = 16$  is the thread-block edge length. Each  $z$ -slice of the grid corresponds to one chromosome; threads within a block collaboratively rasterize that chromosome's  $N$  triangles using barycentric coordinate tests, accumulate per-pixel loss contributions, and reduce the block's partial sum to a single scalar via a parallel tree reduction. One atomic add per block writes the partial sum into the chromosome's loss accumulator. Because all chromosomes are evaluated concurrently, only  $P$  loss scalars are transferred from device to host per generation, keeping PCIe traffic negligible.

**Shared memory utilization.** Triangle vertex and color data for each chromosome are loaded into on-chip shared memory at the start of each block, amortizing global memory latency across all threads in the block. This is critical for performance because every thread in the block accesses the same  $10N$  parameters during rasterization.

**Multi-GPU distribution.** When multiple GPUs are available on a node, the population of  $P$  chromosomes is partitioned evenly across  $G$  devices. Each GPU receives its slice of the chromosome batch, executes the full batched kernel on its slice concurrently via independent CUDA streams, and returns its slice of loss scalars. The host assembles the full fitness array after synchronizing all streams. This design scales the effective batch throughput linearly with the number of GPUs and requires no inter-GPU communication.

### 3.3 MPI Island Model and Ring Migration

To scale beyond a single node and reduce the risk of premature convergence, we adopt an island model [11] in which each MPI rank maintains an independent GA population. Ranks are seeded with distinct random states so that islands explore different regions of the  $[0, 1]^{10N}$  search space from the outset.

**Island independence.** Within a generation, islands operate entirely independently: each rank runs its own GA step and invokes its own batched GPU kernel for fitness evaluation. There is no synchronization barrier between ranks during normal evolution, preserving the parallel efficiency of the design. The total generation budget  $G_{\max}$  is divided evenly across ranks so that the aggregate computational work equals the configured budget regardless of the number of islands.

**Ring migration.** Every  $M$  generations, islands exchange genetic material via a unidirectional ring topology. Rank  $r$  sends its  $m$  best chromosomes (those with lowest loss, residing at the front of



the sorted population) to rank  $(r + 1) \bmod R$  and simultaneously receives  $m$  chromosomes from rank  $(r - 1) \bmod R$ , where  $R$  is the total number of ranks. Migration is implemented as a non-blocking MPI\_Sendrecv so that all ranks exchange simultaneously without deadlock. Received immigrants replace the  $m$  worst chromosomes in the local population and are immediately re-evaluated on the local GPU before the next GA step, ensuring that the fitness array remains consistent.

**Termination across islands.** Any island that satisfies its local termination criterion—stagnation or generation budget—signals all other ranks via MPI\_Allreduce with a logical OR. Once any island halts, all ranks stop and the globally best chromosome is gathered to rank 0 via MPI\_Gather for final output.

## 4 Implementation Details

### 4.1 Configuration System

All hyperparameters are specified through a plain-text key–value configuration file loaded at startup. If no file is provided, the system falls back to a set of built-in defaults. After loading, derived fields—the number of vertex genes  $N_v = 6N$ , color genes  $N_c = 4N$ , and total chromosome length  $N_g = N_v + N_c = 10N$ —are computed automatically. Table 1 lists the full set of configurable parameters and their default values.

**Table 1: Default configuration parameters.**

| Parameter          | Default               | Description                          |
|--------------------|-----------------------|--------------------------------------|
| NTRIANGLES         | 200                   | Triangles per chromosome             |
| POPSIZE            | 30                    | Chromosomes in population            |
| DISCCOUNT          | $\lceil 0.75P \rceil$ | Slots replaced per generation        |
| MAXGENERATIONS     | 0                     | Hard generation limit (0 = disabled) |
| STAGNATIONGENS     | 2000                  | Stagnation window (generations)      |
| STAGNATIONRELATIVE | 0                     | 0 = absolute, 1 = relative           |
| STAGNATIONABSTOL   | $10^{-7}$             | Min absolute improvement             |
| STAGNATIONRELTOL   | $10^{-4}$             | Min relative improvement             |
| CROSSOVERPROB      | 0.95                  | Crossover vs. mutation prob.         |
| ALPHAINIT          | 0.15                  | Fixed initial triangle opacity       |
| LOSSTYPE           | mse                   | Loss function (see Section 3.1)      |
| WMSEPOWER          | 0.5                   | Area-weight exponent $\gamma$        |
| NUMGPUS            | 0                     | GPUs per node (0 = all)              |
| MIGRATIONINTERVAL  | 100                   | Gens between ring migrations         |
| MIGRATIONSIZE      | 1                     | Chromosomes per migration            |
| VISUALISEEVERY     | 10000                 | Gens between PPM snapshots           |

### 4.2 Loss Function Selection

The loss function is selected at runtime via the LOSSTYPE field, which accepts either a string name or a numeric index. The three loss functions evaluated in this work are summarized in Table 2. MSE serves as the primary baseline due to its simplicity and smooth signal. BCE is included to assess sensitivity near channel extremes. WMSE is used to study the effect of primitive-size regularization on reconstruction quality and triangle distribution.

**Table 2: Loss functions and their runtime identifiers.**

| Name | Key   | Primary characteristic           |
|------|-------|----------------------------------|
| MSE  | mse   | Balanced per-pixel error         |
| BCE  | log11 | Aggressive near channel extremes |
| WMSE | wmse  | Penalizes large triangles        |

### 4.3 Triangle Count and Chromosome Dimensionality

The number of triangles  $N$  is the primary driver of both reconstruction fidelity and computational cost. Each additional triangle adds 10 parameters to the chromosome, expands the search space, and increases the per-pixel rasterization work inside the CUDA kernel. We evaluate configurations ranging from  $N = 50$  to  $N = 500$  triangles, yielding chromosome lengths between 500 and 5,000 parameters. Larger triangle counts generally improve reconstruction quality but increase the risk of stagnation as the optimizer must navigate a higher-dimensional landscape.

### 4.4 Stagnation-Based Termination

When MAXGENERATIONS is set to zero, the run is governed entirely by the stagnation criterion. The stagnation counter increments each generation in which the best population loss does not improve by more than the configured tolerance. In absolute mode (STAGNATIONRELATIVE = 0), improvement is declared if

$$\mathcal{L}_{\text{best}}^{(g-1)} - \mathcal{L}_{\text{best}}^{(g)} > \tau_{\text{abs}},$$

and in relative mode (STAGNATIONRELATIVE = 1), if

$$\frac{\mathcal{L}_{\text{best}}^{(g-1)} - \mathcal{L}_{\text{best}}^{(g)}}{\mathcal{L}_{\text{best}}^{(g-1)}} > \tau_{\text{rel}}.$$

Once the counter reaches STAGNATIONGENS consecutive non-improving generations, all islands are signaled to halt via MPI\_Allreduce. This mechanism adapts the run length to the difficulty of the instance, avoiding both premature termination and unnecessary computation on already converged populations.

### 4.5 Experiments

We conduct three categories of experiments to evaluate the system from complementary perspectives.

**Hyperparameter search.** The first category focuses on reconstruction quality as a function of algorithm configuration. We perform a systematic sweep over triangle count  $N \in \{50, 100, 200, 500\}$ , population size  $P \in \{10, 30, 50, 100\}$ , loss function (mse, log11, wmse), and stagnation window  $S \in \{500, 1000, 2000\}$ . Each configuration is run to stagnation-based termination and the final best loss and convergence curve are recorded. This sweep identifies which combinations of primitives, population pressure, and loss signal yield the highest-quality reconstructions, and informs the fixed configuration used in the scaling experiments.

**Strong scaling.** The second category measures parallel efficiency under a fixed problem size. We report results in two regimes. In the *multi-GPU* regime, a single MPI rank runs the GA and the population is partitioned evenly across  $G$  GPUs, each evaluating its

slice of chromosomes concurrently via independent CUDA streams; no ring migration is used. In the *multi-node* regime, each MPI rank operates an independent island on its own GPU(s), with ring migration exchanging elite chromosomes between islands every MIGRATIONINTERVAL generations. Image resolution, triangle count, and population size are held constant across both regimes, and wall-clock time per generation is reported as the number of GPUs and ranks increases from 1 to the maximum available.

**Weak scaling.** The third category measures how well the system maintains constant throughput as the problem grows. As in strong scaling, we report both the multi-GPU and multi-node regimes separately. The workload per GPU is held constant by scaling the image area proportionally to the number of GPUs: if the baseline uses an image of area  $A_0$  on a single GPU, an experiment with  $G$  GPUs targets an image of area  $G \cdot A_0$ , with population size scaled accordingly. Ideal weak scaling yields constant wall-clock time per generation as  $G$  increases.

**Hardware targets and job execution.** All large-scale experiments are conducted on the AiMOS cluster, comprising POWER9 nodes each equipped with multiple NVIDIA V100 GPUs. Jobs are submitted through the Slurm workload manager using sbatch scripts that sweep over configuration files, varying triangle count, population size, number of GPUs, number of MPI ranks, and loss function. Each job runs the GA to termination and writes timing and convergence data—wall-clock time per generation, best loss per generation, and profiler bucket totals—to a structured log file in the job's output directory. Bulk parameter sweeps are executed as sbatch job arrays, one job per configuration, allowing many experimental conditions to run concurrently across the cluster. Results are aggregated from the per-job log files for analysis. Single-node baseline measurements are collected on a commodity x86-64 workstation equipped with an NVIDIA consumer GPU.

## 5 Results

### 5.1 Hyperparameter Search

The optimized hyperparameter was selected on the basis of 120 experiment runs across three independent axes of variation, loss function, cross-over probability, and population size. Out of 5 different types of loss functions, MSE stands out to be most stable. On the basis of the tubular data above which represents Crossover

|             | $\alpha = 0.005$ | $\alpha = 0.010$ | $\alpha = 0.015$ | $\alpha = 0.025$ | $\alpha = 0.040$ |
|-------------|------------------|------------------|------------------|------------------|------------------|
| $cp = 0.50$ | 0.070949         | 0.044332         | 0.028077         | 0.022453         | 0.016697         |
| $cp = 0.70$ | 0.044727         | 0.023519         | 0.017411         | 0.014637         | 0.013178         |
| $cp = 0.85$ | 0.024890         | 0.013668         | 0.011305         | 0.009933         | 0.011210         |
| $cp = 0.95$ | 0.013669         | 0.007911         | 0.007102         | <b>0.006674</b>  | 0.006965         |

**Table 3: Final MSE loss after 100,000 generations across the full  $cp \times \alpha$  grid. Best result in bold.**

Probability ( $cp$ )  $\times$  Alpha Init, it is evident that  $cp$  is the dominant hyperparameter. Changing  $cp$  from 0.5 to 0.95 improves the loss almost 5X at any given alpha. Also if the  $cp$  is highest, the best alpha is not the lower extreme term which is true in real case. Because, when crossover is doing primary driver, you want slightly smaller

mutation perturbations so that recombined genomes are not over modified by large random steps.

| Final MSE loss |          |          |                 |          |
|----------------|----------|----------|-----------------|----------|
| Pop ↓ / Tri →  | 128      | 256      | 512             | 1024     |
| 64             | 0.008919 | 0.008152 | 0.008246        | 0.008325 |
| 128            | 0.007858 | 0.007727 | 0.007182        | 0.007557 |
| 256            | 0.007428 | 0.006203 | 0.006450        | 0.007215 |
| 512            | 0.007176 | 0.006113 | <b>0.005960</b> | 0.006400 |

| Wall time (seconds) |     |      |      |      |
|---------------------|-----|------|------|------|
| Pop ↓ / Tri →       | 128 | 256  | 512  | 1024 |
| 64                  | 115 | 211  | 406  | 858  |
| 128                 | 215 | 409  | 812  | 1722 |
| 256                 | 416 | 814  | 1640 | 3482 |
| 512                 | 832 | 1661 | 3315 | 7248 |

**Table 4: Final MSE loss (left) and wall time in seconds (right) for the population  $\times$  triangle count grid. Best loss in bold.**

Wall time scales proportionally to  $pop \times triangles$ : doubling either dimension doubles the cost. Larger populations consistently reduce loss, with diminishing returns beyond  $pop = 256$ .

Triangle count improves quality only up to 512. At  $tri = 1024$  results are consistently worse than at  $tri = 512$  across all population sizes. This is a dimensionality effect: each additional triangle adds 10 genes to the search space, and the fixed 100,000-generation budget is insufficient to explore the larger space, so the GA makes less relative progress per generation.

Recommended configurations by time budget:

- **Under 1,000 s:**  $pop = 256$ ,  $tri = 256$  (loss = 0.006203, 814 s) — best cost-efficiency.
- **Maximum quality:**  $pop = 512$ ,  $tri = 512$  (loss = 0.005960, 3,315 s).
- **Avoid:**  $pop = 512$ ,  $tri = 1024$  — costs  $2.2\times$  more than  $pop = 512$ ,  $tri = 512$  for a worse result.

So the best choice of hyperparameter configuration is

| LOSS_TYPE          | mse               |
|--------------------|-------------------|
| CROSSOVER_PROB     | 0.95              |
| ALPHA_INIT         | 0.025             |
| POP_SIZE           | 512               |
| N_TRIANGLES        | 512               |
| Expected loss      | $\approx 0.0060$  |
| Wall time (6 GPUs) | $\approx 3,315$ s |

**Table 5: Globally optimal configuration derived from all sweep results.**

This is the rendered image for the best hypermeter selection.

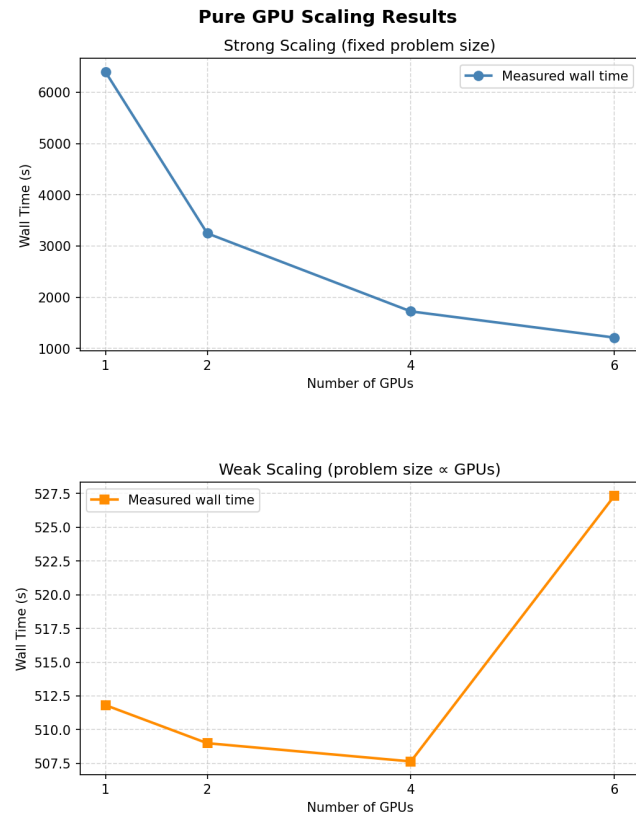
### 5.2 Multi-GPU Scaling

Figure 7 displays the results from the single-node, multi-GPU regime experiments. The top subfigure, depicting the strong scaling experiment, shows a monotonic decrease in wall-clock time from approximately 6,400 seconds on one GPU to approximately 1,200



**Figure 6: Final rendered output for the best hyperparameter configuration (MSE,  $cp=0.95$ ,  $\alpha=0.025$ ,  $pop=512$ ,  $tri=512$ ). Loss = 0.005960.**

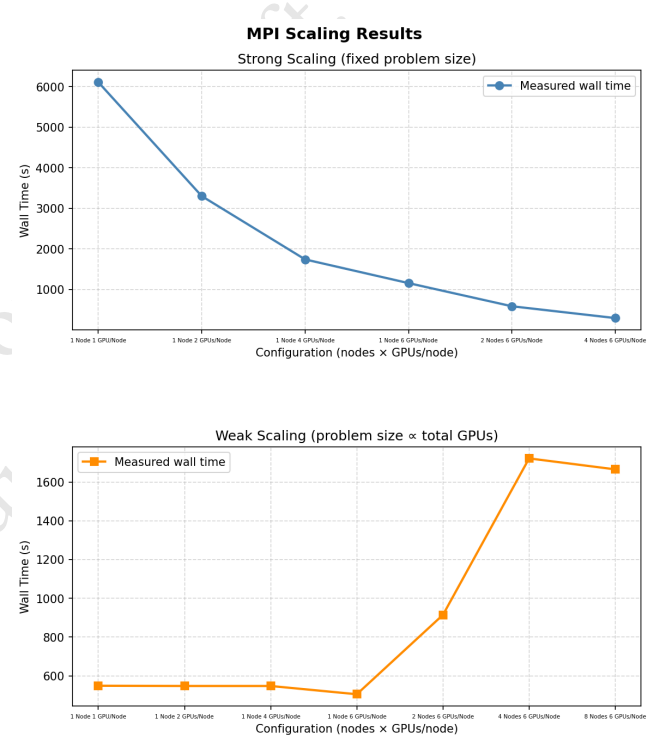
seconds on six GPUs. The bottom subfigure depicts the weak scaling experiment. Execution time decreases slightly between one and four GPUs, from approximately 512 seconds to 508 seconds, before rising to 527 seconds in the six-GPU trial.



**Figure 7: Strong and weak scaling experimental results for single-node, multi-GPU experiments.**

### 5.3 Multi-Node MPI Scaling

Figure 8 displays the experimental results from the multi-node, multi-GPU regime. The top subfigure depicts the strong scaling results, showing a monotonic decrease in wall-clock time from approximately 6,100 seconds on one node with one GPU to approximately 300 seconds on four nodes each with six GPUs. The bottom subfigure displays the weak scaling results. Execution time remains roughly constant on a single node across one to four GPUs per node, then falls slightly in the single-node, six-GPU case. Wall time then rises monotonically for multi-node configurations, reaching a peak of 1,722 seconds at four nodes and six GPUs per node, before decreasing slightly to 1,665 seconds at eight nodes and six GPUs per node.



**Figure 8: Strong and weak scaling experimental results for multi-node experiments.**

### 5.4 Timing Breakdown

Tables 6 and 7 report the per-bucket timing breakdown for the strong and weak scaling experiments respectively, decomposed into the render bucket—which encompasses batched CUDA kernel execution—and the optimize bucket—which encompasses selection, crossover, mutation, and population management on the host.

In the strong scaling experiments, Table 6, render time dominates across every configuration, accounting for 98.8% of wall time in the single-GPU baseline. Notably, optimize time remains nearly constant at approximately 74 seconds across all single node multi-GPU configurations, but rises sharply to 214 seconds and 139 seconds in the one-node, two-GPU one-node, four-GPU MPI configurations



respectively before returning to approximately 71 seconds at higher GPU counts. In the two-node and four-node configurations, both render and optimize times decrease proportionally as the generation budget is divided across ranks.

In the weak scaling experiments, Table 7, render time remains nearly flat across all single-node configurations. Ranging from 437 seconds down to 431 seconds, consistent with the proportional increase in image area being absorbed by the additional GPUs. A notable discontinuity appears in the optimize bucket for the MPI single-node configurations at one to four GPUs per node, where optimize time is approximately 114–116 seconds, roughly 60% higher than the non-MPI baseline of approximately 74 seconds. This overhead disappears at six GPUs per node, where optimize time returns to approximately 71 seconds. In the multi-node MPI configurations, render time rises substantially as image area scales with the number of ranks, reaching 1,639 seconds at four nodes and six GPUs per node.

## 6 Analysis

### 6.1 Multi-GPU Strong Scaling

The strong scaling results for the single-node, multi-GPU regime demonstrate near-linear speedup as the number of GPUs increases from one to six. As shown in Figure 7, wall-clock time falls from approximately 6,400 seconds on a single GPU to approximately 1,211 seconds on six GPUs, yielding a speedup of 5.3 $\times$  against an ideal of 6 $\times$ . The sub-linear speedup is consistent with Amdahl's Law as the optimize bucket—comprising selection, crossover, and mutation—remains entirely serial and is unaffected by GPU count. As Table 6 confirms, optimize time holds nearly constant at approximately 74 seconds across all multi-GPU configurations, while render time contracts from 6,319 seconds to 1,134 seconds. Consequently, the render fraction of wall time decreases from 98.8% at one GPU to 93.8% at six GPUs, meaning the serial optimize fraction grows from 1.2% to 6.2% and will increasingly limit achievable speedup as GPU counts scale further.

### 6.2 Multi-GPU Weak Scaling

The weak scaling experiment scales image area proportionally to the number of GPUs so that each GPU's workload remains constant. As shown in Figure 7 and Table 7, wall time is nearly flat between one and four GPUs—decreasing marginally from 511 to 507 seconds—before rising slightly to 527 seconds in the six-GPU trial. The render bucket mirrors this behavior, remaining between 432 and 452 seconds across all configurations, while optimize time is stable at approximately 74 seconds throughout. The marginal increase at six GPUs likely reflects load imbalance at GPU partition boundaries when image dimensions are not evenly divisible by the number of devices, introducing idle threads at partition edges. Overall, the six-GPU trial incurs only a 3% increase in wall time relative to the single-GPU baseline, confirming that the batched kernel design scales rendering throughput in close proportion to the number of GPUs.

### 6.3 MPI Strong Scaling

The multi-node strong scaling experiment extends the GPU scaling results across cluster nodes under the island model with ring

migration. As Figure 8 shows, fixing the total generation budget at  $G_{\max} = 100,000$  and dividing it evenly across ranks, wall-clock time decreases monotonically from approximately 6,111 seconds at one node and one GPU to approximately 294 seconds at four nodes each with six GPUs, yielding a speedup of 20.8 $\times$  over a total resource increase of 24 $\times$  and a parallel efficiency of approximately 87%.

Table 6 reveals two notable features of the MPI strong scaling breakdown. First, the optimize time spikes to 214 seconds and 139 seconds at one node with two and four GPUs respectively, before returning to approximately 71 seconds at six GPUs per node and continuing to decrease to 18 seconds at four nodes. This anomaly at intermediate GPU counts on a single node likely reflects thread contention or synchronization overhead between the MPI runtime and the CUDA streams when multiple GPUs are managed by a single rank; the overhead dissipates once the GPU count reaches six, suggesting that the scheduler achieves better overlap at that configuration. Second, as ranks increase beyond a single node, both render and optimize times decrease proportionally, as each rank executes a smaller fraction of the total generation budget. The slight increase in final loss observed at higher rank counts—0.006–0.007 versus 0.005–0.006 for the single-node baseline—is consistent with the reduced per-island generation budget and reflects a quality-throughput trade-off inherent to the island model under a fixed total budget.

### 6.4 MPI Weak Scaling

The MPI weak scaling experiment reveals more complex behavior than the single-node GPU case. On a single node, Table 7 shows that render time remains nearly constant at approximately 431–433 seconds across one to six GPUs per node, consistent with the GPU weak scaling results. However, the optimize bucket exhibits an unexpected elevation of approximately 114–116 seconds for the one-to-four-GPU-per-node MPI configurations, compared with approximately 74 seconds in the equivalent non-MPI runs. This overhead disappears at six GPUs per node, where optimize time returns to approximately 71 seconds. This pattern mirrors the anomaly observed in the MPI strong scaling results and similarly suggests contention between the MPI runtime and CUDA stream management at intermediate GPU counts on a single node.

As the experiment scales to multiple nodes, render time increases substantially—from 432 seconds at one node and six GPUs to 1,639 seconds at four nodes and six GPUs—because each island evaluates a proportionally larger image as area scales as  $R \times A_0$ , where  $R$  is the number of ranks. This coupling between aggregate image size and per-node render cost distinguishes the MPI weak scaling regime from the single-node case and explains the superlinear growth in wall time visible in Figure 8. The subsequent slight decrease in wall time at eight nodes and six GPUs per node, despite a further doubling of image area, suggests that the additional intra-node parallelism at that configuration partially offsets the increased per-pixel rasterization cost. These results highlight that weak scaling efficiency in the island model is governed not only by per-GPU rendering workload but also by the aggregate image size each island must evaluate—a coupling that must be carefully accounted for in experimental design and resource allocation.



**Table 6: Timing breakdown for strong scaling experiments. Render time dominates across all configurations; the optimize fraction grows as GPU count increases, consistent with Amdahl's Law.**

| Config                        | Render (s) | Optimize (s) | Total (s) | Render (%) |
|-------------------------------|------------|--------------|-----------|------------|
| <i>Multi-GPU, single node</i> |            |              |           |            |
| 1 GPU                         | 6318.77    | 73.86        | 6392.63   | 98.8%      |
| 2 GPU                         | 3169.47    | 74.13        | 3243.60   | 97.7%      |
| 4 GPU                         | 1644.24    | 74.60        | 1718.84   | 95.7%      |
| 6 GPU                         | 1133.65    | 74.41        | 1208.06   | 93.8%      |
| <i>Multi-node MPI</i>         |            |              |           |            |
| 1 Node 1 GPU                  | 6031.60    | 76.30        | 6107.91   | 98.8%      |
| 1 Node 2 GPUs                 | 3081.65    | 214.28       | 3295.93   | 93.5%      |
| 1 Node 4 GPUs                 | 1593.79    | 138.68       | 1732.46   | 92.0%      |
| 1 Node 6 GPUs                 | 1082.08    | 71.40        | 1153.48   | 93.8%      |
| 2 Nodes 6 GPUs                | 546.64     | 35.26        | 581.90    | 93.9%      |
| 4 Nodes 6 GPUs                | 272.20     | 17.52        | 289.72    | 94.0%      |

**Table 7: Timing breakdown for weak scaling experiments. Render time remains nearly constant in the single-node multi-GPU regime, confirming proportional throughput scaling. Render time rises substantially in multi-node trials as each island evaluates a proportionally larger image.**

| Config                        | Image       | Render (s) | Optimize (s) | Total (s) | Render (%) |
|-------------------------------|-------------|------------|--------------|-----------|------------|
| <i>Multi-GPU, single node</i> |             |            |              |           |            |
| 1 GPU                         | 200 × 200   | 437.26     | 73.85        | 511.10    | 85.6%      |
| 2 GPU                         | 283 × 283   | 434.13     | 73.95        | 508.08    | 85.4%      |
| 4 GPU                         | 400 × 400   | 432.68     | 73.88        | 506.56    | 85.4%      |
| 6 GPU                         | 490 × 490   | 451.49     | 74.36        | 525.84    | 85.9%      |
| <i>Multi-node MPI</i>         |             |            |              |           |            |
| 1 Node 1 GPU                  | 200 × 200   | 433.23     | 113.91       | 547.13    | 79.2%      |
| 1 Node 2 GPUs                 | 283 × 283   | 430.17     | 115.44       | 545.60    | 78.8%      |
| 1 Node 4 GPUs                 | 400 × 400   | 430.21     | 114.96       | 545.16    | 78.9%      |
| 1 Node 6 GPUs                 | 490 × 490   | 431.84     | 71.27        | 503.11    | 85.8%      |
| 2 Nodes 6 GPUs                | 693 × 693   | 835.56     | 71.16        | 906.72    | 92.2%      |
| 4 Nodes 6 GPUs                | 980 × 980   | 1639.14    | 70.47        | 1709.60   | 95.9%      |
| 8 Nodes 6 GPUs                | 1386 × 1386 | 1610.25    | 34.33        | 1644.58   | 97.9%      |

## 7 Conclusions

We have presented a fully parallel, multi-level Genetic Algorithm system for triangle-based image vectorization, spanning CUDA thread-level parallelism, multi-GPU partitioning, and multi-node MPI execution under an island model with ring migration. The system achieves substantial throughput gains over a serial baseline: strong scaling experiments demonstrate speedups of 5.3× on a single node with six GPUs and approximately 20.8× across four nodes each equipped with six GPUs, with parallel efficiencies of 88% and 87% respectively. Weak scaling experiments confirm near-constant wall time in the single-node, multi-GPU regime, with a modest 3% overhead at six GPUs attributable to partition boundary effects at non-divisible image resolutions.

The per-bucket timing results reported in Tables 6 and 7 confirm that the render bucket dominates wall time across every configuration, validating the design priority of accelerating fitness evaluation through batched CUDA kernels and shared memory caching of triangle parameters. The optimize bucket—comprising selection, crossover, and mutation—remains approximately constant in absolute time regardless of GPU count, and its growing fractional contribution to wall time as rendering accelerates represents the primary barrier to further single-node speedup, consistent with Amdahl's Law. An anomalous elevation of optimize time at intermediate GPU counts in the single-node MPI configurations suggests runtime contention between the MPI layer and CUDA stream management, and warrants further profiling to isolate and mitigate.

The island model introduces an inherent quality–throughput trade-off under a fixed total generation budget: distributing the

**Table 8: Team member contributions.**

| Team Member      | Contributions                                                                                                                                                                                                                                                                                              |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mohammed Elkoumy | <ul style="list-style-type: none"> <li>• Lead the team and proposed the main ideas</li> <li>• Explored some variations of optimizers</li> <li>• Implemented multi-gpu and MPI parallelization at scale</li> <li>• Ran bulk of experiments with bash scripts</li> </ul>                                     |
| Quinten Pienaar  | <ul style="list-style-type: none"> <li>• Conducted experimentation on dual-annealing vs genetic algorithm as the primary optimizer to implement</li> <li>• Synthesized raw data into figures and tables displayed in results section</li> <li>• Wrote results, analysis and conclusion sections</li> </ul> |
| Mahir Mobarrat   | <ul style="list-style-type: none"> <li>• Base CUDA kernel implementation for initial experimentation.</li> <li>• Hyperparameter optimization analysis.</li> </ul>                                                                                                                                          |
| Ganwar Harsh     | <ul style="list-style-type: none"> <li>• Conducted a comprehensive literature review of the topic</li> <li>• Verified and validated experimental data and results</li> <li>• Contributed to manuscript writing, flowchart design, and performed a thorough review</li> </ul>                               |

budget across ranks reduces per-island exploration depth, yielding slightly higher final loss values at large rank counts. Ring migration mitigates premature convergence by periodically injecting elite chromosomes from neighboring islands, sustaining population diversity without requiring a global synchronization barrier. Future work should investigate adaptive migration schedules and variable

per-island budgets that allocate more generations to islands showing continued improvement, potentially recovering solution quality without sacrificing throughput.

As noted in the implementation, a promising direction for future work is the incorporation of dual annealing as an alternative or complementary local search operator. Dual annealing combines simulated annealing with a fast Cauchy distribution for large jumps, which could accelerate escape from local minima that the current mutation operators traverse only slowly. Hybridizing the island GA with a local annealing phase applied to elite chromosomes between migration events may yield both faster convergence and higher final reconstruction quality.

## References

- [1] David E. Goldberg. 1989. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- [2] John H. Holland. 1992. Genetic Algorithms. *Scientific American* 267, 1 (1992), 66–73. doi:10.1038/scientificamerican0792-66
- [3] Teng Hu et al. 2024. SuperSVG: Superpixel-based Scalable Vector Graphics Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [4] Roger Johansson. 2008. Genetic Programming: Evolution of Mona Lisa. <https://rogerjohansson.blog/2008/12/07/genetic-programming-evolution-of-mona-lisa/>.
- [5] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. 2020. Differentiable Vector Graphics Rasterization for Editing and Learning. *ACM Transactions on Graphics* 39, 6 (2020). doi:10.1145/3414685.3417871
- [6] Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, and Humphrey Shi. 2022. Towards Layer-Wise Image Vectorization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 16314–16323.
- [7] Thomas Porter and Tom Duff. 1984. Compositing Digital Images. *SIGGRAPH Comput. Graph.* 18, 3 (jan 1984), 253–259. doi:10.1145/964965.808606
- [8] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. 2019. Regularized Evolution for Image Classifier Architecture Search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4780–4789. doi:10.1609/aaai.v33i01.33014780
- [9] Peter Selinger. 2003. Potrace: a polygon-based tracing algorithm. <https://potrace.sourceforge.net/>.
- [10] Karl Sims. 1991. Artificial Evolution for Computer Graphics. In *Proceedings of the 18th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. 319–328. doi:10.1145/122718.122752
- [11] Darrell Whitley, Timothy Starkweather, and D'Ann Fuquay. 1989. Scheduling Problems and Traveling Salesmen: The Genetic Edge Recombination Operator. In *Proceedings of the Third International Conference on Genetic Algorithms (ICGA)*. Morgan Kaufmann, 133–140.

## A Appendix

### Team Contributions

Table 8 summarizes the contributions of each team member to the project.